

3.2 MLIR中的类型转换

MLIR 的类型系统

MLIR 的类型系统包括 MLIR 内置类型和方言相关类型(包括 LLVM 方言类型)。



MLIR 的类型转换

MLIR 类型转换的作用便是 桥接不同类型之间的差异。

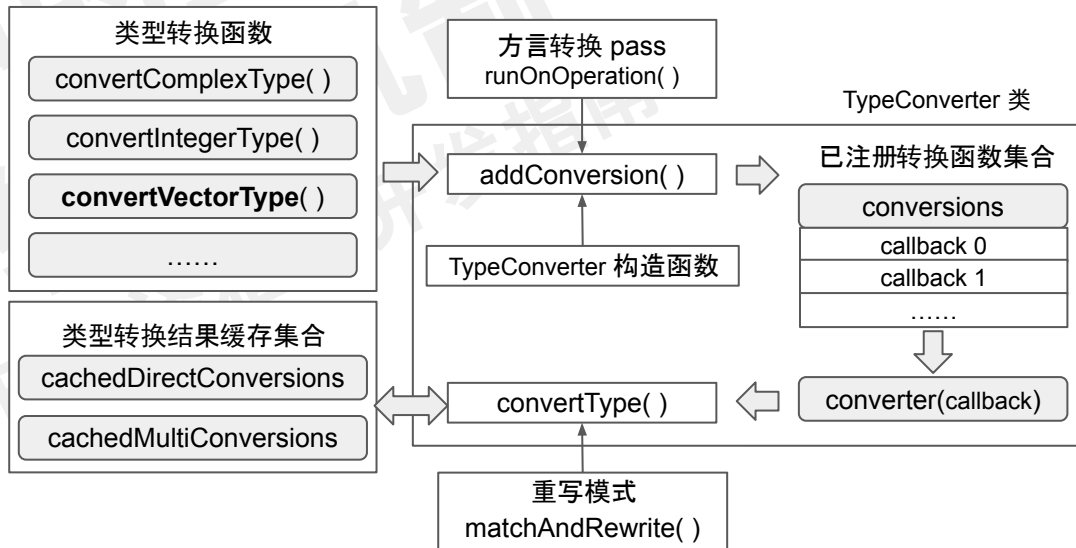
类型转换函数定义了将给定的源类型转换为一个或多个证结果类型的转换规则。

TypeConverter 类提供了常用的 类型转换相关接口, 如 addConversion() 和 convertType()。

类型转换函数定义了将给定的源类型转换为一个或多个结果类型的转换规则。

```
struct ConvertNVGPUToNVVMPass
: public impl::ConvertNVGPUToNVVMPassBase<ConvertNVGPUToNVVMPass> {
...
void runOnOperation() override {
  RewritePatternSet patterns(&getContext());
  LLVMTypeConverter converter(&getContext());
...
  converter.addConversion([&](nvgpu::DeviceAsyncTokenType type) -> Type {
    return converter.convertType(IntegerType::get(type.getContext(), 32));
  });
...
};

void registerConversion(ConversionCallbackFn callback) {
  conversions.emplace_back(std::move(callback));
  cachedDirectConversions.clear();
  cachedMultiConversions.clear();
}
```



向量类型转换函数

convertVectorType()

vector<4x8x16xf32>  !llvm.array<4 x [8 x <16xf32>]>

```
FailureOr<Type> LLVMTypeConverter::convertVectorType(VectorType type) const {  
    auto elementType = convertType(type.getElementType());  
    if (!elementType) return {};  
    if (type.getShape().empty()) return VectorType::get({1}, elementType);  
    Type vectorType = VectorType::get(type.getShape().back(), elementType,  
                                       type.getScalableDims().back());  
  
    ...  
    auto shape = type.getShape();  
    for (int i = shape.size() - 2; i >= 0; --i)  
        vectorType = LLVM::LLVMArrayType::get(vectorType, shape[i]);  
    return vectorType;  
}
```

```
Type LLVMTypeConverter::convertFunctionType(FunctionType type) const {  
    return LLVM::LLVMPointerType::get(type.getContext());  
}
```

0 维向量: vector<f32> -> vector<1xf32>
一维向量: vector<nxf32> -> vector<nxf32>
多维向量: vector<n0xn1...xnkxT>
-> !llvm.array<n0x[n1x[...<nkxT>]]>

类型转换过程

```
LogicalResult
TypeConverter::convertType(Type t, SmallVectorImpl<Type> &results) {
...
for (ConversionCallbackFn &converter : llvm::reverse(conversions)) {
    if (std::optional<LogicalResult> result = converter(t, results, ...) ) {
        ...
        auto newTypes = ArrayRef<Type>(results).drop_front(currentCount);
        if (newTypes.size() == 1)
            cachedDirectConversions.try_emplace(t, newTypes.front());
        else
            cachedMultiConversions.try_emplace(t, llvm::to_vector<2>(newTypes));
        return success();
    }
}
return failure();
}
```

